

Dynamic RC-WTCTP Project Crashing

K13 Pemodelan Matematika

May 8, 2026

1 Pendahuluan

Dalam manajemen proyek modern, penyelesaian proyek tepat waktu dengan biaya minimal merupakan salah satu tantangan utama yang dihadapi oleh manajer proyek. Proyek sering kali menghadapi ketidakpastian, keterbatasan kapasitas sumber daya, dan tekanan tenggat waktu (**deadline**). Untuk mengatasi masalah ini, dikembangkan dua konsep optimisasi penting: **Resource-Constrained Project Scheduling Problem** (RCPSP) dan **Time-Cost Trade-off Problem** (TCTP).

RCPSP berfokus pada penjadwalan aktivitas proyek dengan memperhatikan batasan ketergantungan antar-aktivitas (**precedence constraints**) dan keterbatasan kapasitas sumber daya harian yang tersedia. Di sisi lain, TCTP mengasumsikan bahwa durasi pengerjaan suatu aktivitas dapat dipangkas (**project crashing**) dengan cara mengalokasikan sumber daya tambahan, yang konsekuensinya akan meningkatkan biaya proyek.

Secara umum, terdapat dua skenario **project crashing** yang dibahas dalam laporan ini:

1. **Skenario Baseline:** Menghadapi masalah **crashing** ketika biaya pemotongan durasi harian (**crash cost per day**) dan batas durasi minimum (**minimum duration**) untuk setiap aktivitas telah diketahui secara eksplisit dan bernilai konstan. Model ini dikembangkan menggunakan paradigma pemrograman batasan (**Constraint Programming**) melalui Google OR-Tools CP-SAT.
2. **Skenario Novel:** Menghadapi masalah **crashing** ketika biaya pemotongan durasi dan batas durasi minimum tidak diketahui secara langsung. Sebaliknya, manajer proyek hanya memiliki tuas kontrol berupa alokasi harian sumber daya manusia, yang dapat dipercepat melalui dua mekanisme: penambahan tenaga kerja (**overcrowding**) dan penambahan jam kerja lembur (**overtime**). Hubungan non-linier antara alokasi sumber daya tambahan dengan durasi aktivitas dimodelkan menggunakan fungsi produksi Cobb-Douglas guna menangkap efek penurunan efisiensi marjinal (**diminishing returns**). Model ini diselesaikan menggunakan pemrograman linier integer campuran (MILP) berbasis diskretisasi serta algoritma genetika (metaheuristik).

Laporan ini menyajikan perumusan model matematika lengkap, penjelasan data, asumsi, justifikasi variabel, serta metode penyelesaian untuk kedua skenario di atas secara terstruktur.

2 Baseline Model (Skenario 1)

Model baseline ini merumuskan suatu masalah optimisasi berupa **Resource-Constrained Time-Cost Trade-off Problem** (RC-TCTP) dinamis pada proyek dengan durasi diskrit dan biaya pemotongan durasi (**crashing cost**) yang linier.

2.1 Deskripsi Data

Model baseline ini dibangun menggunakan tiga berkas data utama yang saling berkaitan:

1. **Data Aktivitas** (`activity_data_v3.json`): Menentukan daftar aktivitas proyek, durasi normal ($d_i^{\max}$), durasi minimum setelah *crashing* ($d_i^{\min}$), biaya pemangkasan harian (C_i), hubungan ketergantungan antar-aktivitas (*precedence*), serta status pengerjaan tugas saat ini.
2. **Kapasitas Sumber Daya** (`resource_capacity_v3.json`): Menentukan batas kapasitas harian maksimum ($U_k^{\max}$) untuk setiap jenis sumber daya k .
3. **Kebutuhan Sumber Daya** (`resource_requirements_v3.json`): Menentukan kebutuhan harian aktivitas i terhadap sumber daya k (yaitu $U_{i,k}$) ketika aktivitas tersebut sedang aktif berjalan.

Contoh format dan sampel data terintegrasi untuk Skenario 1 dapat dilihat pada tabel di bawah ini:

Nama Aktivitas	Precedence	$d_i^{\min}$	$d_i^{\max}$	C_i (USD/hari)	Sumber Daya (Kebutuhan)
Bids & Contracts	-	7 hari	10 hari	\$60.00	General Management (1), Project Management (1)
Grading & Permits	Bids & Contracts	7 hari	10 hari	\$70.00	General Management (1), Survey Crew (1), Grading Contractor (2)
Site Work	Grading & Permits	5 hari	7 hari	\$30.00	Labor Crew (3), Grading Contractor (2), Survey Crew (1)

2.2 Asumsi dan Limitasi Model

Formulasi model baseline didasarkan pada beberapa asumsi dan limitasi berikut:

- **Linieritas Biaya Crashing:** Pengurangan durasi suatu aktivitas diasumsikan memiliki biaya tambahan konstan per hari. Total biaya crashing untuk suatu tugas adalah hasil kali dari jumlah hari pemotongan dengan biaya harian.
- **Precedence Finish-to-Start:** Hubungan ketergantungan antar-aktivitas yang didukung hanya tipe **Finish-to-Start** (FS) tanpa adanya **lag/lead** waktu.
- **Non-Preemptive:** Aktivitas yang sedang dikerjakan tidak dapat diinterupsi atau diberhentikan sementara hingga benar-benar selesai.
- **Sunk Cost:** Biaya crashing yang terjadi pada masa lalu (sudah selesai) dianggap sebagai biaya hangus (**sunk cost**) dan diabaikan dari fungsi tujuan optimisasi aktif.

2.3 Notasi

Variabel	Satuan	Deskripsi
Himpunan		
I	-	Aktivitas, di-indeks oleh i
I_0	-	Aktivitas sehingga $e_i \leq T_0$ (sudah selesai)
I_1	-	Aktivitas sehingga $s_i \geq T_0$ (belum mulai)
E	-	Ketergantungan antara aktivitas, di-indeks oleh (i, j)
K	-	Sumber daya, di-indeks oleh k
Parameter		
$s_i^{(0)}$	hari	Hari mulai aktual (sebelum <i>crashing</i>) aktivitas i
$e_i^{(0)}$	hari	Hari akhir aktual (sebelum <i>crashing</i>) aktivitas i
$d_i^{(\max)}$	hari	Durasi normal aktivitas i
$d_i^{(\min)}$	hari	Durasi minimum aktivitas i
C_i	Rp/hari	Harga harian untuk <i>crash</i> aktivitas i
$U_{i,k}$	Rp/hari	Keperluan harian sumber daya k untuk aktivitas i
$U_k^{(\max)}$	Rp/hari	Kapasitas harian sumber daya k
$T_{\max}$	hari	Hari tenggat proyek (<i>deadline</i>)
T_0	hari	Hari mulai <i>crashing</i> proyek
B	hari	Anggaran total untuk <i>crashing</i>
Variabel Keputusan		
s_i	hari	Hari mulai aktivitas i
e_i	hari	Hari akhir aktivitas i

2.4 Formulasi Batasan (Constraints)

2.4.1 Batasan Durasi

Pertama, perhatikan bahwa $d_i := e_i - s_i$ menyatakan jumlah hari yang di-*crash*. Karena *crashing* adalah tindakan terbatas (tidak mungkin membuat aktivitas selesai secara instan), diperlukan batasan untuk d_i , yakni:

$$d_i^{\min} \leq e_i - s_i \leq d_i^{\max} \quad \forall i \in I.$$

Jadi, durasi pengerjaan suatu aktivitas harus lebih dari *minimum task duration* dan setidaknya selama *normal task duration*.

2.4.2 Batasan Ketergantungan

Dalam manajemen, proyek dapat digambarkan sebagai jaringan proyek (I, E) dengan I adalah semua aktivitas dan E adalah ketergantungan (*dependencies*) antara aktivitas. Terdapat empat jenis ketergantungan: *finish-to-start* (FS), *finish-to-finish* (FF), *start-to-start* (SS), dan *start-to-*

finish (SF). Berdasarkan data kami peroleh, kami fokuskan diri hanya pada jenis ketergantungan *finish-to-start* (FS) yang paling sederhana. Ini menghasilkan batasan

$$s_i \geq e_j \quad \forall (i, j) \in E.$$

Maka, sebuah aktivitas hanya bisa dimulai setelah semua aktivitas pendahulunya (berdasarkan jaringan proyek yang diberikan) selesai.

2.4.3 Batasan Sumber Daya

Dalam masalah *resource-constrained* (di *project-scheduling* atau *time-cost tradeoff*), suatu sumber daya tidak bisa digunakan semena-mena. Pada tiap waktu pelaksanaan, jumlah suatu sumber daya tertentu yang digunakan tidak boleh melebihi kapasitas yang tersedia:

$$\sum_{i \in I} U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < e_i\} \leq U_k^{\max}, \quad \forall k \in K, \forall j \in I$$

Secara matematis, pengecekan cukup dilakukan di setiap awal aktivitas (tidak perlu setiap satuan waktu) untuk menghemat komputasi. Di *OR-Tools*, implementasi mudah hanya dengan variabel *interval CP-SAT* dan fungsi pembantu *AddCumulative*.

2.4.4 Batasan Dinamis

Berikutnya, khususnya untuk *project crashing* (bukan TCTP secara umum), diperlukan batasan untuk aktivitas-aktivitas yang akan di-*crash*. Untuk aktivitas yang belum dilaksanakan:

$$s_i \geq T_0, \quad \forall i \in I_1.$$

Untuk aktivitas yang sedang dilaksanakan:

$$\begin{aligned} s_i &= s_i^{(0)}, \quad \forall i \in I_0^C \cap I_1^C. \\ e_i &\geq T_0, \end{aligned}$$

Untuk aktivitas yang sudah selesai:

$$\begin{aligned} s_i &= s_i^{(0)}, \quad \forall i \in I_0. \\ e_i &= e_i^{(0)}, \end{aligned}$$

2.5 Formulasi Objektif

2.5.1 Cost-Driven

Di sini, difokuskan aspek *cost* dari TCTP, sehingga ingin meminimumkan biaya total:

$$\min \sum_{i \in I_0^C} C_i (d_i^{\max} - (e_i - s_i)).$$

Jadi, diperlukan batasan keras agar tenggat terpenuhi:

$$s_{n+1} \leq T_{\max}.$$

2.5.2 Time-Driven

Di sini, difokuskan aspek *time* dari TCTP, sehingga ingin meminimumkan waktu pengerjaan:

$$\min s_{n+1}.$$

Ini menjadi mirip dengan masalah RCPSP biasa. Jadi, diperlukan batasan anggaran saja:

$$\sum_{i \in I} C_i(d_i^{\max} - (e_i - s_i)) \leq B.$$

2.5.3 Multi-Objektif

Di sini, kita gabungkan kedua perspektif di atas untuk menghasilkan masalah multi-objektif yang sesuai untuk TCTP:

$$\min \left(s_{n+1}, \sum_{i \in I_0^C} C_i(d_i^{\max} - (e_i - s_i)) \right).$$

2.5.4 Bonus-Penalty Driven

Pemodelan *time-cost tradeoff* tidak perlu dengan pendekatan multi-objektif seperti di atas. Penggabungannya bisa mempertahankan fungsi objektif tunggal dengan menggunakan metrik penalti harian c_{late} dan bonus harian c_{early} . Ini menghasilkan fungsi objektif berikut:

$$\min \sum_{i \in I_0^C} C_i(d_i^{\max} - (e_i - s_i)) + c_{\text{late}} \max(0, s_{n+1} - T_{\max}) - c_{\text{early}} \max(0, T_{\max} - s_{n+1}).$$

2.6 Metode Penyelesaian

Model baseline diselesaikan menggunakan **Google OR-Tools CP-SAT Solver**. CP-SAT dipilih karena menyediakan variabel interval (**IntervalVar**) secara native yang mempermudah representasi durasi tugas dan memiliki algoritma propagasi batasan kumulatif (**AddCumulative**) yang sangat efisien untuk memecahkan masalah penjadwalan dengan batasan sumber daya berkapasitas terbatas.

Integer Scaling: Karena solver CP-SAT hanya menerima nilai koefisien berupa bilangan bulat (integer), parameter biaya crashing riil (cc_a) yang berbentuk desimal dikalikan dengan faktor skala $S = 10^{\{\text{max_dp}\}}$ (di mana max_dp adalah presisi desimal maksimum dari data biaya). Koefisien tujuan ter-skala dihitung dengan $\text{coeff}_a = \lfloor cc_a \cdot S \rfloor$. Setelah diperoleh solusi optimal, total biaya riil diperoleh dengan membagi kembali nilai objektif solver dengan S .

3 Novel Model (Skenario 2)

Di sini, kami memformulasikan model TCTP baru. Secara khusus, dibuat model RC-WTCTP berbasis **workforce** (bukan durasi seperti CTCP atau mode seperti DTCTP) dengan fungsi biaya **endogenous** (tidak **exogenous** seperti pada TCTP umumnya) yang diturunkan dari model utilitas Cobb-Douglas.

3.1 Deskripsi Data

Model ini menggunakan berkas data sebagai berikut:

1. **Data Aktivitas** (`task_table.json`): Berisi daftar seluruh aktivitas (I), level outline, durasi baseline, tanggal mulai dan selesai, serta dependensi (*precedence*) dengan nilai **lag/lead**-nya.
2. **Data Sumber Daya** (`resource_table.json`): Berisi daftar jenis tenaga kerja (K), kapasitas maksimal harian ($U_k^{\max}$ dalam persen), dan tarif upah standar per jam (r_k).
3. **Data Alokasi Tugas** (`assignment_table.json`): Menghubungkan aktivitas dengan tenaga kerja yang ditugaskan (K_i), mencakup usaha kerja baseline ($W_{i,k}$ dalam jam) dan persentase alokasi harian baseline ($U_{i,k}$).

Contoh format dan sampel data terintegrasi untuk Skenario 2 ditunjukkan pada tabel-tabel di bawah ini:

ID	Nama Aktivitas (<code>task_table</code>)	Durasi Baseline	Outline Level	Predecessors
2	Receive notice to proceed and sign contract	3 hari	2	-
3	Submit bond and insurance documents	2 hari	2	2FS
4	Prepare and submit project schedule	4 hari	2	3FS

ID	Nama Sumber Daya (<code>resource_table</code>)	Max Units (Kapasitas)	Standard Rate (Tarif)
1	G.C. General Management	100% (1 orang)	\$120.00/jam
2	G.C. Project Management	100% (1 orang)	\$95.00/jam
6	G.C. Labor Crew	300% (3 orang)	\$35.00/jam

Nama Aktivitas (<code>assignment_table</code>)	Nama Sumber Daya	Work (Usaha)	Units (Alokasi)
Submit bond and insurance documents	G.C. Project Management	16 jam	100%
Submit bond and insurance documents	G.C. General Management	4 jam	25%
Prepare and submit project schedule	G.C. Scheduler	16 jam	100%

3.2 Asumsi dan Limitasi Model

Formulasi model Cobb-Douglas didasarkan pada beberapa asumsi realistis berikut:

- **Diminishing Returns:** Menambah jumlah pekerja pada suatu aktivitas (**overcrowding**) atau memperpanjang jam kerja (**overtime**) akan mempercepat pengerjaan aktivitas, namun dengan efisiensi marjinal yang menurun. Hal ini dimodelkan dengan eksponen elastisitas $\alpha, \beta \in (0, 1)$ pada fungsi Cobb-Douglas.
- **Koordinasi dan Kelelahan:** Overcrowding menyebabkan hilangnya efisiensi karena peningkatan beban koordinasi antar-pekerja. Overtime menurunkan produktivitas pekerja karena kelelahan (**fatigue**).
- **Tarif Lembur Lebih Tinggi:** Jam kerja lembur (τ) dikenakan tarif upah yang lebih tinggi daripada jam reguler (dikali multiplier upah lembur $r'_k = \text{ot_mult} \cdot r_k$).
- **Non-Preemptive:** Pengerjaan tugas bersifat kontinu dari hari mulai hingga selesai (non-preemptive).

3.3 Notasi

Variabel	Satuan	Deskripsi
Himpunan		
I	-	Himpunan semua aktivitas
I_0	-	Aktivitas sehingga $e_i \leq T_0$ (sudah selesai)
I_1	-	Aktivitas sehingga $s_i \geq T_0$ (belum mulai)
E_{FS}, E_{SS}, E_{FF}	-	Relasi ketergantungan <i>finish-to-start</i> , <i>start-to-start</i> , dan <i>finish-to-finish</i>
K	-	Sumber daya, di-indeks oleh k
Parameter		
$W_{i,k}$	jam	Usaha kerja baseline SDM k untuk aktivitas i
$U_{i,k}$	-	Alokasi harian baseline SDM k untuk aktivitas i (proporsi dari 8 jam)
C_k	unit	Kapasitas SDM k yang tersedia per hari
r_k	Rp/jam	Tarif reguler SDM k
r'_k	Rp/jam	Tarif lembur SDM k , dengan $r'_k \geq r_k$
α	-	Eksponen Cobb–Douglas untuk overcrowding ($0 < \alpha < 1$)
β	-	Eksponen Cobb–Douglas untuk overtime ($0 < \beta < 1$)
c_{late}	Rp/hari	Penalti keterlambatan
c_{early}	Rp/hari	Bonus penyelesaian awal
c_{ind}	Rp/hari	Biaya overhead proyek tidak langsung harian
δ_{ij}	hari	Lag/lead antara aktivitas i dan j
T_0	hari	Hari saat ini peninjauan proyek
Variabel Keputusan		
$x_{i,k}$	-	Pengali overcrowding SDM k untuk aktivitas $i \in V_2$
$\tau_{i,k}$	jam/hari	Lama overtime harian SDM k untuk aktivitas $i \in V_2$
s_i	hari	Hari mulai aktivitas i

3.4 Formulasi Model

3.4.1 Motivasi

Berdasarkan data penjadwalan *baseline*, durasi normal aktivitas i dapat dihitung dari kebutuhan kerja per tugas sebagai

$$d_i^{(0)} = \max_{k \in K_i} \frac{W_{i,k}}{8U_{i,k}},$$

sedangkan biaya *baseline* untuk tenaga kerja k pada tugas i adalah

$$z_{i,k}^{(0)} = W_{i,k} r_k.$$

Dari rumus di atas, terlihat bahwa kita bisa mensimulasikan *crashing* dengan mengubah nilai durasi kerja harian 8 dan usaha kerja $U_{i,k}$. Jadi, kita bisa memperkenalkan suatu variabel pengali **tenaga kerja** $x_{i,k}$ dan variabel aditif **jam lembur** $\tau_{i,k}$. Jika kita mengimplementasikan variabel *crashing* ini secara naif dengan langsung menggabungkannya di rumus durasi aktivitas i untuk SDM k , akan diperoleh

$$d'_{i,k} = \frac{W_{i,k}}{(8 + \tau_{i,k})(x_{i,k}U_{i,k})}.$$

Namun, perhatikan bahwa rumusan naif ini mengakibatkan total biaya sumber daya tidak berbeda dari *baseline*:

$$z_{i,k} = d'_{i,k} \cdot x_{i,k}U_{i,k} \cdot (8 + \tau_{i,k}) \cdot r_{i,k} = W_{i,k}r_{i,k} = z_{i,k}^{(0)}.$$

3.4.2 Cobb-Douglas

Di dunia sempurna, ini mungkin benar: menggandakan jumlah pekerja akan memaruhkan durasi kerja, sehingga biaya total akan sama. Tetapi secara realistis, ini tentu tidak masuk akal, sebab terdapat *inefficiency* yang mengakibatkan Untuk mengatasi ini, diperkenalkan **fungsi produksi Cobb-Douglas** untuk memodelkan fenomena *diminishing returns*, yang berbunyi:

$$Y(L, K) = AL^\alpha K^\beta$$

dengan Y adalah output produksi total, L adalah input tenaga kerja (*labor*), dan K adalah input modal (*capital*). Lalu, A adalah faktor produktivitas total, $\alpha \in (0, 1)$ menyatakan elastisitas tenaga kerja, dan $\beta \in (0, 1)$ menyatakan elastisitas modal. Di kasus *project crashing*, kita punya bahwa jumlah sumber daya $x_{i,k}U_{i,k}$ mewakili tenaga kerja L dan durasi total kerja $8 + \tau_{i,k}$ mewakili modal K . Jadi, kita bisa menggantikan rumus durasi aktivitas i untuk SDM k setelah *crashing* menjadi

$$d_{i,k}(x_{i,k}, \tau_{i,k}) = \frac{W_{i,k}}{A_{i,k}(x_{i,k}U_{i,k})^\alpha (8 + \tau_{i,k})^\beta}.$$

Sekarang, dengan memilih konstanta $A_{i,k} = U_{i,k}^{1-\alpha} 8^{1-\beta}$ sehingga durasi baru bernilai sama dengan *baseline* ketika $x_{i,k} = 1$ and $\tau_{i,k} = 0$, maka durasi yang sudah di-*crash* dapat disederhanakan menjadi

$$d_{i,k}(x_{i,k}, \tau_{i,k}) = \underbrace{\frac{W_{i,k}}{8U_{i,k}}}_{\text{baseline}} \cdot \underbrace{\left(\frac{1}{x_{i,k}}\right)^\alpha}_{\text{overwork}} \cdot \underbrace{\left(\frac{8}{8 + \tau_{i,k}}\right)^\beta}_{\text{overtime}}.$$

sehingga durasi aktivitas i saja menjadi

$$d_i = \max_{k \in K_i} d_{i,k}(x_{i,k}, \tau_{i,k})$$

Dengan ini, ingat kembali bahwa durasi pengerjaan, usaha harian, dan biaya harian sudah berubah.

1. Durasi “efektif” pengerjaan aktivitas i oleh SDM k berubah menjadi $d_{i,k}^*(x_{i,k}, \tau_{i,k})$.
2. Total usaha SDM k pada aktivitas i adalah $x_{i,k} U_{i,k}$.
3. Biaya harian mencakup upah reguler (8 jam dengan tarif r_k) dan upah lembur ($\tau_{i,k}$ jam dengan tarif lembur r'_k), sehingga gaji totalnya adalah $8r_k + \tau_{i,k} r'_k$.

Dengan mengalikan ketiga suku tersebut, diperoleh total biaya $z_{i,k}(x_{i,k}, \tau_{i,k})$ baru untuk SDM k pada aktivitas i adalah

$$\begin{aligned} z_{i,k}(x_{i,k}, \tau_{i,k}) &= d_{i,k}(x_{i,k}, \tau_{i,k}) \cdot x_{i,k} U_{i,k} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= \frac{W_{i,k}}{8U_{i,k}} \cdot \left(\frac{1}{x_{i,k}}\right)^\alpha \cdot \left(\frac{8}{8 + \tau_{i,k}}\right)^\beta \cdot x_{i,k} U_{i,k} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= W_{i,k} \cdot x_{i,k}^{-\alpha} \cdot x_{i,k} \cdot \left(\frac{8 + \tau_{i,k}}{8}\right)^{-\beta} \cdot \frac{8 + \tau_{i,k}}{8} \cdot \frac{1}{8 + \tau_{i,k}} \cdot (8r_k + \tau_{i,k} r'_k) \\ &= \underbrace{W_{i,k} r_k}_{\text{baseline}} \cdot \underbrace{x_{i,k}^{1-\alpha}}_{\text{overman}} \cdot \underbrace{\left(\frac{8 + \tau_{i,k}}{8}\right)^{1-\beta}}_{\text{overtime}} \cdot \underbrace{\frac{8 + \frac{r'_k}{r_k} \tau_{i,k}}{8 + \tau_{i,k}}}_{\text{extra wage}}. \end{aligned}$$

Persamaan biaya ini secara elegan memodelkan **diminishing returns**: karena $1 - \alpha, 1 - \beta < 1$, peningkatan *overcrowding* ($x_{i,k} > 1$) maupun *overtime* ($\tau_{i,k} > 0$) akan menghasilkan total biaya yang lebih tinggi daripada *baseline*, menyelesaikan masalah yang kita miliki sebelumnya.

3.4.3 Estimasi Parameter

α : digitalcommons.unl.edu/cgi/viewcontent.cgi?article=1012&context=constructiondiss

β : www.long-intl.com/articles/loss-of-labor-productivity/

3.4.4 Contoh

Perhatikan ilustrasi tugas di bawah ini:

Submit shop drawings and order long lead items - roofing Roofing Contractor Management 80h 100%

Secara normal, pengerjaan tugas ini membutuhkan **Roofing Contractor Management** untuk bekerja 8 jam per hari selama 10 hari (total 80 jam kerja *baseline*).

1. **Mekanisme Overtime:** Jika pekerja diminta untuk lembur $\tau = 1$ jam per hari (total 9 jam kerja per hari), dengan efek kelelahan $\beta = 0.3$, durasi efektifnya menjadi $d = 10 \cdot \left(\frac{8}{9}\right)^{0.3} \approx 9.6$ hari. Upah jam lembur dihitung dengan tarif r'_k lebih tinggi.
2. **Mekanisme Overcrowding:** Jika kita menambah satu pekerja lagi ($x = 2.0$) dengan efek koordinasi $\alpha = 0.6$, durasi baru menjadi $d' = 10 \cdot \frac{1}{2^{0.6}} \approx 6.6$ hari. Jumlah hari-pekerja meningkat menjadi $6.6 \cdot 2 = 13.2$ hari-orang, yang berarti biaya labor total naik sebesar $2^{1-0.6} = 2^{0.4} \approx 1.32$ kali biaya baseline.

3.5 Formulasi Batasan

Pertama, berdasarkan hasil yang diperoleh di Bagian 3.4, diulang definisi durasi efektif oleh SDM k pada aktivitas i adalah

$$d_{i,k}(x_{i,k}, \tau_{i,k}) := \frac{W_{i,k}}{8U_{i,k}} \cdot \left(\frac{1}{x_{i,k}}\right)^\alpha \cdot \left(\frac{8}{8 + \tau_{i,k}}\right)^\beta,$$

sedangkan biaya efektif untuk aktivitas i dan SDM k adalah

$$z_{i,k}(x_{i,k}, \tau_{i,k}) = W_{i,k} r_k \cdot x_{i,k}^{1-\alpha} \cdot \left(\frac{8 + \tau_{i,k}}{8}\right)^{1-\beta} \cdot \frac{8 + \frac{r'_k}{r_k} \tau_{i,k}}{8 + \tau_{i,k}}.$$

Serupa, didefinisikan hari akhir suatu aktivitas i sebagai:

$$e_i := s_i + \max_{k \in K} d_{i,k}(x_{i,k}, \tau_{i,k}).$$

Ketiga variabel di atas ini **bukan** variabel keputusan, melainkan hanya variabel pembantu untuk kami formulasi model di bagian-bagian berikut dengan lebih mudah. Cukup mensubstitusikan dua ekspresi di atas setiap ada munculnya $d_{i,k}$, $z_{i,k}$, atau e_i di seluruh rumusan berikutnya untuk menghilangkan mereka.

3.5.1 Batasan Ketergantungan

Di sini, model akan diperumum untuk mendukung tiga jenis ketergantungan: *finish-to-start* (FS), *finish-to-finish* (FF), dan *start-to-start* (SS). Ini menghasilkan batasan berikut:

$$\begin{aligned} s_j &\geq e_i + \delta_{ij}, & \forall (i, j) \in E_{\text{FS}}, \\ s_j &\geq s_i + \delta_{ij}, & \forall (i, j) \in E_{\text{SS}}, \\ e_j &\geq e_i + \delta_{ij}, & \forall (i, j) \in E_{\text{FF}}. \end{aligned}$$

Serupa, ketergantungan *start-to-finish* bisa ditambahkan jika perlu pada data.

3.5.2 Batasan Sumber Daya

Seperti di Bagian 2, sifat *resource-constrained* dari RC-TCTP diberikan oleh batasan sumber daya. Untuk model ini, diformulasikan batasannya sebagai berikut:

$$\sum_{i \in I} U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < s_i + d_{i,k}\} \leq U_k^{\max}, \quad \forall k \in K, \forall j \in I$$

Sekali lagi, pengecekan cukup dilakukan di setiap awal aktivitas (tidak perlu setiap satuan waktu) untuk menghemat komputasi. Tetapi selebihnya, perhatikan bahwa di sini batas atas adalah $s_i + d_{i,k}$, bukan e_i . Perbedaan ini berarti bahwa keputusan alokasi sumber daya bukan dilakukan pada level aktivitas, melainkan pada level sumber daya-nya sendiri. Jadi, suatu sumber daya bisa memutuskan untuk berpindah ke aktivitas lain jika sudah selesai dengan tugas mereka di aktivitas sekarang.

3.5.3 Batasan Dinamis

Terakhir, diperlukan batasan untuk aktivitas-aktivitas yang akan atau tidak akan di-*crash*. Untuk aktivitas yang belum dilaksanakan:

$$s_i \geq T_0, \quad \forall i \in I_1,$$

dan perhitungan waktu kerja total menggunakan yang orisinal:

$$W_{i,k} = W_{i,k}^{(0)}, \quad \forall i \in I_1.$$

Untuk aktivitas yang sedang dilaksanakan, kita menghitung suatu rasio $p_{i,k}$ yaitu bagian dari durasi *baseline* yang sudah dikerjakan. Dengan itu, kita definisikan waktu kerja total efektifnya adalah

$$W_{i,k} = W_{i,k}^{(0)}(1 - p_{i,k}), \quad \forall i \in I_0^C \cap I_1^C.$$

Selain itu, diperlukan seperti biasa

$$s_i = s_i^{(0)}, \quad \forall i \in I_0^C \cap I_1^C.$$

Untuk aktivitas yang sudah selesai:

$$\begin{aligned} s_i &= s_i^{(0)}, \\ x_{i,k} &= 1, \quad \forall i \in I_0, \\ \tau_{i,k} &= 0, \end{aligned}$$

dan perhitungan waktu kerja total menggunakan yang orisinal:

$$W_{i,k} = W_{i,k}^{(0)}, \quad \forall i \in I_0.$$

3.6 Formulasi Objektif

3.6.1 Cost-Driven

Di sini, difokuskan aspek *cost* dari TCTP, sehingga ingin meminimumkan biaya total:

$$\min \sum_{i \in I_0^C} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}).$$

Jadi, diperlukan batasan keras agar tenggat terpenuhi:

$$s_{n+1} \leq T_{\max}.$$

3.6.2 Time-Driven

Di sini, difokuskan aspek *time* dari TCTP, sehingga ingin meminimumkan waktu pengerjaan:

$$\min s_{n+1}.$$

Ini menjadi mirip dengan masalah RCPSP biasa. Jadi, diperlukan batasan anggaran saja:

$$\sum_{i \in I} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}) \leq B.$$

3.6.3 Multi-Objektif

Di sini, kita gabungkan kedua perspektif di atas untuk menghasilkan masalah multi-objektif yang sesuai untuk TCTP:

$$\min \left(s_{n+1}, \sum_{i \in I_0^C} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}) \right).$$

3.6.4 Bonus-Penalty Driven

Pemodelan *time-cost tradeoff* tidak perlu dengan pendekatan multi-objektif seperti di atas. Penggabungannya bisa mempertahankan fungsi objektif tunggal dengan menggunakan metrik penalti harian c_{late} dan bonus harian c_{early} . Ini menghasilkan fungsi objektif berikut:

$$\min \sum_{i \in I_0^C} \sum_{k \in K} z_{i,k}(x_{i,k}, \tau_{i,k}) + c_{\text{late}} \max(0, s_{n+1} - T_{\max}) - c_{\text{early}} \max(0, T_{\max} - s_{n+1}).$$

3.7 Metode Penyelesaian Skenario 2

Model Cobb-Douglas asli merupakan model **MINLP (Mixed-Integer Non-Linear Programming)** yang non-konveks karena persamaan durasi dan biaya mengandung eksponen pecahan. Untuk menyelesaikan model ini secara efisien dan andal, digunakan dua pendekatan:

3.7.1 MILP berbasis Diskretisasi Grid

Metode ini mendiskretkan ruang pencarian kontinu pengali overcrowding $x_{i,k}$ dan overtime harian $\tau_{i,k}$ menjadi beberapa titik grid tertentu:

- $x_g \in \{1.0, 1.25, 1.5, 1.75, 2.0\}$
- $\tau_g \in \{0.0, 1.0, 2.0, 3.0, 4.0\}$

Definisikan variabel keputusan biner baru $\xi_{i,k}^{m,n} \in \{0,1\}$ yang bernilai 1 jika SDM k pada tugas i memilih titik grid overcrowding ke- m dan overtime ke- n . Dengan diskretisasi ini, nilai durasi $d_{\{i,m,n\}}$ dan biaya harian labor cost $_{i,m,n}$ untuk masing-masing kombinasi grid dihitung terlebih dahulu sebelum optimisasi (**precomputed**). Seluruh konstrain durasi dan biaya menjadi fungsi linier terhadap variabel biner $\xi_{i,k}^{m,n}$:

$$d_i = \sum_m \sum_n \xi_{i,k}^{m,n} \cdot d_{\{i,m,n\}}.$$

Model ini kemudian dimodelkan menggunakan **Pyomo** dan diselesaikan menggunakan solver MILP komersial/open-source seperti **CBC** atau **HiGHS** hingga mencapai jaminan solusi optimal global dalam hitungan detik.

3.7.2 Pendekatan Metaheuristik (Genetic Algorithm)

Sebagai alternatif pembanding untuk ruang pencarian kontinu tanpa diskretisasi, diimplementasikan algoritma genetika (GA) menggunakan pustaka **pymoo** di Python:

- **Representasi Kromosom:** Variabel keputusan (x, τ) dikodekan langsung sebagai vektor bilangan real.
- **Fungsi Penalti:** Karena GA sulit menangani batasan secara langsung, kendala precedence dan kapasitas sumber daya ditambahkan sebagai penalti kuadratis ke dalam fungsi objektif jika terjadi pelanggaran (**precedence violation penalty**).
- **Operator Genetika:** Menggunakan operator seleksi turnamen, persilangan SBX (*Simulated Binary Crossover*), dan mutasi PM (*Polynomial Mutation*).

Meskipun metode GA dapat menangani fungsi Cobb-Douglas asli tanpa diskretisasi, ia tidak memberikan jaminan optimalitas global dan membutuhkan waktu komputasi yang lebih lama untuk konvergensi dibandingkan dengan pendekatan MILP diskret.

4 Hybrid Model (Skenario 3)

4.1 Deskripsi Data

Model Hybrid menggunakan struktur data yang sama dengan Skenario 2, yakni data aktivitas (`task_table.json`), data sumber daya (`resource_table.json`), dan alokasi tugas (`assignment_table.json`). Pengguna tidak perlu menyediakan estimasi biaya pemotongan harian (C_i) secara langsung, karena parameter tersebut akan dihitung secara endogen dari data internal organisasi.

4.2 Asumsi dan Limitasi Model

Formulasi model hybrid didasarkan pada beberapa asumsi berikut:

- **Linearisasi Lokal:** Hubungan non-linier antara alokasi sumber daya dengan durasi proyek didekati secara linier melalui perhitungan **crash slope** lokal antara kondisi baseline dengan kondisi akselerasi maksimal.
- **Batas Kendali Maksimal:** Ditentukan batas overcrowding maksimum ($x_{\max} = 2.0$) dan batas overtime harian maksimum ($\tau_{\max} = 2.0$ jam/hari) sebagai kondisi batas percepatan proyek yang realistis.
- **Sunk Cost:** Biaya masa lalu diabaikan dalam optimisasi dinamis.

4.3 Langkah Preprocessing Data

Untuk menjembatani realisme data Skenario 2 dengan kecepatan komputasi Skenario 1, dilakukan preprocessing menggunakan fungsi Cobb-Douglas untuk setiap aktivitas i dan jenis SDM $k \in K_i$:

1. **Durasi Normal ($d_i^{(\max)}$):**

$$d_i^{(\max)} = \max_{k \in K_i} \left[\frac{W_{i,k}}{8U_{i,k}} \right]$$

2. **Biaya Normal (Z_i^{base}):**

$$Z_i^{\text{base}} = \sum_{k \in K_i} W_{i,k} \cdot r_k$$

3. **Durasi Minimum ($d_i^{(\min)}$):**

$$d_i^{(\min)} = \max_{k \in K_i} \left[\frac{W_{i,k}}{8U_{i,k}} \cdot \left(\frac{1}{x_{\max}} \right)^\alpha \cdot \left(\frac{8}{8 + \tau_{\max}} \right)^\beta \right]$$

4. **Biaya Crashing Maksimum (Z_i^{crash}):**

$$Z_i^{\text{crash}} = \sum_{k \in K_i} z_{i,k}(x_{\max}, \tau_{\max})$$

dengan:

$$z_{i,k}(x_{\max}, \tau_{\max}) = W_{i,k} r_k \cdot x_{\max}^{1-\alpha} \cdot \left(\frac{8 + \tau_{\max}}{8} \right)^{1-\beta} \cdot \frac{8 + \frac{r'_k}{r_k} \tau_{\max}}{8 + \tau_{\max}}$$

5. **Crash Slope (C_i):**

$$C_i = \begin{cases} \frac{Z_i^{\text{crash}} - Z_i^{\text{base}}}{d_i^{(\max)} - d_i^{(\min)}} & \text{jika } d_i^{(\max)} > d_i^{(\min)} \\ 0 & \text{jika } d_i^{(\max)} = d_i^{(\min)} \end{cases}$$

4.4 Integrasi ke CP-SAT Solver

Nilai $d_i^{(\max)}$, $d_i^{(\min)}$, dan C_i yang dihitung pada tahap preprocessing digunakan langsung sebagai parameter input untuk model penjadwalan CP-SAT. Model ini meminimalkan biaya crashing total:

$$\min \sum_{i \in I_0^C} C_i \left(d_i^{(\max)} - (e_i - s_i) \right) + c_{\text{late}} \max(0, s_{n+1} - T_{\max}) - c_{\text{early}} \max(0, T_{\max} - s_{n+1})$$

dengan batasan-batasan sebagai berikut:

- **Batasan Waktu:**

$$e_i = s_i + d_i, \quad \forall i \in I$$

- **Batasan Durasi:**

$$d_i^{(\min)} \leq e_i - s_i \leq d_i^{(\max)}, \quad \forall i \in I$$

- **Batasan Precedence:**

$$s_i \geq e_j, \quad \forall (i, j) \in E$$

- **Batasan Kapasitas Sumber Daya:**

$$\sum_{i \in I} U_{i,k} \cdot \mathbb{1}\{s_i \leq s_j < e_i\} \leq U_k^{(\max)}, \quad \forall k \in K, \forall j \in I$$

- **Batasan Dinamis:** Untuk aktivitas yang belum dilaksanakan:

$$s_i \geq T_0, \quad \forall i \in I_1.$$

Untuk aktivitas yang sedang dilaksanakan:

$$\begin{aligned} s_i &= s_i^{(0)}, \\ e_i &\geq T_0, \end{aligned} \quad \forall i \in I_0^C \cap I_1^C.$$

Untuk aktivitas yang sudah selesai:

$$\begin{aligned} s_i &= s_i^{(0)}, \\ e_i &= e_i^{(0)}, \end{aligned} \quad \forall i \in I_0.$$

Karena seluruh konstrain dan fungsi tujuan di atas bersifat linier, model ini dapat diselesaikan oleh CP-SAT secara optimal global dalam hitungan milidetik.

5 Kesimpulan dan Perbandingan Model

Karakteristik	Model Baseline (Skenario 1)	Model Novel (Skenario 2)	Model Hybrid (Skenario 3)
Input Biaya	Eksplisit diketahui per hari crashing per tugas (\$/hari).	Ditentukan secara endogen dari tarif reguler/lembur SDM (\$/jam).	Dihitung secara endogen melalui preprocessing Cobb-Douglas, lalu di-linearisasi per tugas.
Tuas Akselerasi	Langsung memotong hari durasi tugas.	Menambah pekerja (overcrowding) & menambah jam kerja lembur (overtime).	Menggunakan batas $x_{\max}$ dan $\tau_{\max}$ pada preprocessing untuk memotong durasi.
Sifat Efisiensi	Efisiensi konstan (biaya linier terhadap waktu pemangkasan).	Efisiensi menurun (diminishing returns) akibat koordinasi & kelelahan.	Efisiensi non-linier didekati dengan linearisasi lokal (crash slope) per tugas.
Tipe Precedence	Finish-to-Start (FS) sederhana tanpa lag.	Finish-to-Start (FS), Start-to-Start (SS), Finish-to-Finish (FF) dengan lag.	Finish-to-Start (FS) sederhana tanpa lag.
Metode Solver	Constraint Programming (OR-Tools CP-SAT).	MILP (Pyomo + CBC) via Diskretisasi Grid & Metaheuristik (pymoo GA).	Constraint Programming (OR-Tools CP-SAT) setelah preprocessing.

Secara ringkas, jika manajer proyek memiliki estimasi biaya crashing langsung dan mengabaikan efek kelelahan kerja, **Model Baseline** adalah pilihan tercepat. Untuk alokasi proyek taktis yang memperhatikan kelelahan pekerja secara dinamis dan memiliki hubungan precedence kompleks, **Model Novel Cobb-Douglas** memberikan presisi yang tinggi. Namun, jika manajer proyek menginginkan solusi yang realistis berbasis data perusahaan tetapi membutuhkan kecepatan komputasi yang sangat tinggi (milidetik) dengan jaminan solusi optimal global, **Model Hybrid** menawarkan solusi jalan tengah terbaik dengan memadukan keunggulan preprocessing Cobb-Douglas dan efisiensi solver CP-SAT.